SPE-226998-MS

Abstract Factory Generator Design Pattern for Managing Multiple Seismic Propagators on Different Computational Architectures

H. AlSalem and T. Tonellot-Laurent, Saudi Aramco, Dhahran, Eastern Province, Saudi Arabia

This paper was prepared for presentation at the Middle East Oil, Gas and Geosciences Show (MEOS GEO), Manama, Bahrain, 16 – 18 September 2025.

Design Patterns are frequently used models in Object-Oriented Software Design. Their primary purpose is to leverage successful software design practices. Design Patterns also play a crucial role in enhancing the understanding of application design and minimizing communication barriers by employing universally recognized terms for implemented solutions. They typically define a problem and provide a solution, often backed by examples. It is essential to rely on common sense when determining specific implementations, even when applying well-established Design Patterns. Regular critique and thorough, thoughtful testing are crucial in assessing a design's effectiveness and quality [1], [2].

Identifying the optimal Design Pattern for a given software project often proves to be a rigorous and multifaceted endeavor. This challenge arises from the inherent disconnect between software requirements and the explicit characteristics of Design Patterns. Requirements rarely convey clear indicators of potential design issues, leaving developers to rely on their expertise and experience to anticipate challenges and select an appropriate solution. Furthermore, the selection process demands not only a thorough understanding of the Design Patterns themselves but also an ability to align them seamlessly with the project's technical and functional goals. This underscores the importance of a meticulous approach in bridging the gap between abstract concepts and practical application in software engineering.

The Abstract Factory design pattern addresses the instantiation of concrete classes through two distinct interfaces. The first interface is tasked with creating a family of related and dependent products, while the second interface is responsible for creating specific concrete products. By utilizing only the declared interfaces, the client remains unaware of the concrete families and products that are being created. This abstraction ensures a flexible and scalable system, allowing for easy modification and extension without affecting the client code.

Introduction

Design Patterns are frequently used models in Object-Oriented Software Design. Their primary purpose is to leverage successful software design practices. Design Patterns also play a crucial role in enhancing the understanding of application design and minimizing communication barriers by employing universally recognized terms for implemented solutions. They typically define a problem and provide a solution, often backed by examples. It is essential to rely on common sense when determining specific implementations,

even when applying well-established Design Patterns. Regular critique and thorough, thoughtful testing are crucial in assessing a design's effectiveness and quality [1], [2].

Identifying the optimal Design Pattern for a given software project often proves to be a rigorous and multifaceted endeavor. This challenge arises from the inherent disconnect between software requirements and the explicit characteristics of Design Patterns. Requirements rarely convey clear indicators of potential design issues, leaving developers to rely on their expertise and experience to anticipate challenges and select an appropriate solution. Furthermore, the selection process demands not only a thorough understanding of the Design Patterns themselves but also an ability to align them seamlessly with the project's technical and functional goals. This underscores the importance of a meticulous approach in bridging the gap between abstract concepts and practical application in software engineering.

The Abstract Factory design pattern addresses the instantiation of concrete classes through two distinct interfaces. The first interface is tasked with creating a family of related and dependent products, while the second interface is responsible for creating specific concrete products. By utilizing only the declared interfaces, the client remains unaware of the concrete families and products that are being created. This abstraction ensures a flexible and scalable system, allowing for easy modification and extension without affecting the client code.

Managing multiple seismic kernels across diverse computational architectures presents a significant challenge that exponentially increases. This complexity necessitates the use of the Abstract Factory Design Pattern. The pattern allows us to consider various combinations of kernel physics, such as acoustic and elastic, and anisotropy, including isotropic, vertically transverse isotropy (VTI), and tilted transverse isotropy (TTI). Furthermore, we need to account for different computer architectures, including compute processing units (CPUs) and graphics processing units (GPUs). Creating an object for each combination of these factors is time-consuming and significantly increases the number of lines in our codebase. While the conventional Abstract Factory Design Pattern [3] can help mitigate this issue, it doesn't address the issue entirely, as we still need to include a substantial amount of repetitive code. In this paper, we present the solution to generate Abstract Factories for managing multiple configurations of seismic propagators.

Problem Description: Managing Multi-Architecture, Multi-Physics Wave Equation Kernels

Seismic imaging (e.g., full-waveform inversion, reverse-time migration) requires solving time-domain wave-equation PDEs on large 3D grids. These wave-propagation solvers dominate computational cost and are heavily optimized for modern HPC platforms [4]. In practice, one often needs specialized code paths for each hardware and physical model, leading to a combinatorial explosion of kernels. For example, supporting multi-core CPUs versus GPUs (CUDA/HIP/SYCL) often requires distinct low-level implementations, while physics models include acoustic, viscoacoustic, and elastic formulations, each with isotropic or anisotropic (VTI/HTI/TTI) variants. Each combination (architecture × PDE model × anisotropy) demands a separate, hand-tuned kernel, greatly complicating software design.

Key factors contributing to this complexity include:

- **Hardware diversity:** Multi-core CPUs (with OpenMP/SIMD) and many-core accelerators (GPUs, etc.) each benefit from custom data layouts and parallelization strategies.
- **Physics variants:** Acoustic, elastic, and viscoacoustic wave equations must be supported, with both isotropic and anisotropic (VTI, HTI, TTI) media. Elastic and anisotropic models have more unknown fields and more complex stencils.
- **Discretization schemes:** High-order finite-difference or finite-element discretizations are continually evolving, producing further variants of the stencil kernels.

- **Coupled solvers:** Transitioning from an acoustic to an elastic or attenuative model often requires rewriting kernels or adding terms, which must then be revalidated for each platform.

The net result is a vast matrix of code variants. Maintaining and synchronizing all of them is a major engineering burden. Optimizing for one scenario (e.g., CPU-based isotropic acoustics) may break others, and bug fixes must be propagated across many versions. This scenario exemplifies the classic performance–portability–maintainability trade-off [5], [6]. In particular, highly intrusive optimizations (e.g. GPU-specific tuning) "rarely strike the right performance vs productivity balance, as the software becomes less maintainable and extensible" [4]. Traditional workflows—where developers choose a physics/discretization and then hand-optimize for a target architecture—often produce code that is not easily ported, maintained, or extended. In other words, hand-written, hand-optimized kernels tend to be brittle and fragmented.

The multiplicity of kernels imposes costs at every stage. First, **development effort** grows linearly (or worse) with the number of supported cases. For example, implementing a 3D elastic VTI solver is far more complex than its isotropic acoustic counterpart. Second, **consistency and correctness** become hard to guarantee. Ensuring that a bug fix or algorithmic improvement propagates correctly to every variant is error-prone and time-consuming. Third, **performance portability** is elusive. Code optimized for GPUs (e.g. using explicit memory coalescing or shared-memory blocking) can differ drastically from CPU-optimized code, making it difficult to verify that both yield the same results. Indeed, many simulation codes exhibit a substantial portability gap: an implementation that runs correctly on one architecture may not even compile, let alone perform well, on another. This heterogeneity complicates debugging and hampers confidence in cross-platform accuracy.

Moreover, **reproducibility** suffers when kernels proliferate. If a research group's code only runs on a specific GPU with a particular physics option, other groups cannot easily reproduce its results on different hardware or with a different implementation. Poor maintainability thus directly impacts scientific productivity and reproducibility. In practice, teams may accumulate dozens of low-level kernels (in C/C++, CUDA, Fortran, etc.) that are hard to read, test, and extend. Changes in hardware (new GPU architectures) or scientific requirements (new anisotropy models) force further maintenance cycles, diverting effort from innovation.

Abstract Factory Design Pattern for Seismic Kernels

To comprehend the Abstract Factory Generator, let us explore how conventional programming techniques were utilized to program the propagators utilizing the Abstract Factory Design Pattern. This creational pattern functions as an interface for creating families of related or dependent objects without explicitly specifying their concrete classes. It enables developers to define a blueprint for multiple factories, each capable of generating objects conforming to a shared theme. This pattern proves particularly beneficial when managing intricate object structures that require adaptability to various environments or configurations. By harnessing the power of the Abstract Factory, code becomes more modular, scalable, and easier to maintain, as it encapsulates the decisions regarding object creation.

The Abstract Factory pattern, shown as Unified Modeling Language (UML) diagram in Figure 1, comprises Abstract Products, Concrete Products, an Abstract Factory, and Concrete Factories. Abstract Products define interfaces for a set of distinct but related products that form a product family. Concrete Products are various implementations of Abstract Products grouped by variants. Each Abstract Product must be implemented in all given variants. The Abstract Factory interface declares methods for creating each Abstract Product. Concrete Factories implement these creation methods. Each concrete factory corresponds to a specific variant of products and creates only those variants. Although concrete factories instantiate concrete products, their creation methods must return corresponding abstract products. This way, the client application that uses a factory remains decoupled from the specific variant of the product it receives. The

client can work with any concrete factory or product variant as long as it communicates with their objects via abstract interfaces. The following describes when this pattern can be applied [7]:

- The system should be independent of how its products are created, composed, and represented.
- The system should be configured with one of the multiple families of products.
- A family of related and dependent products is designed to be used together.

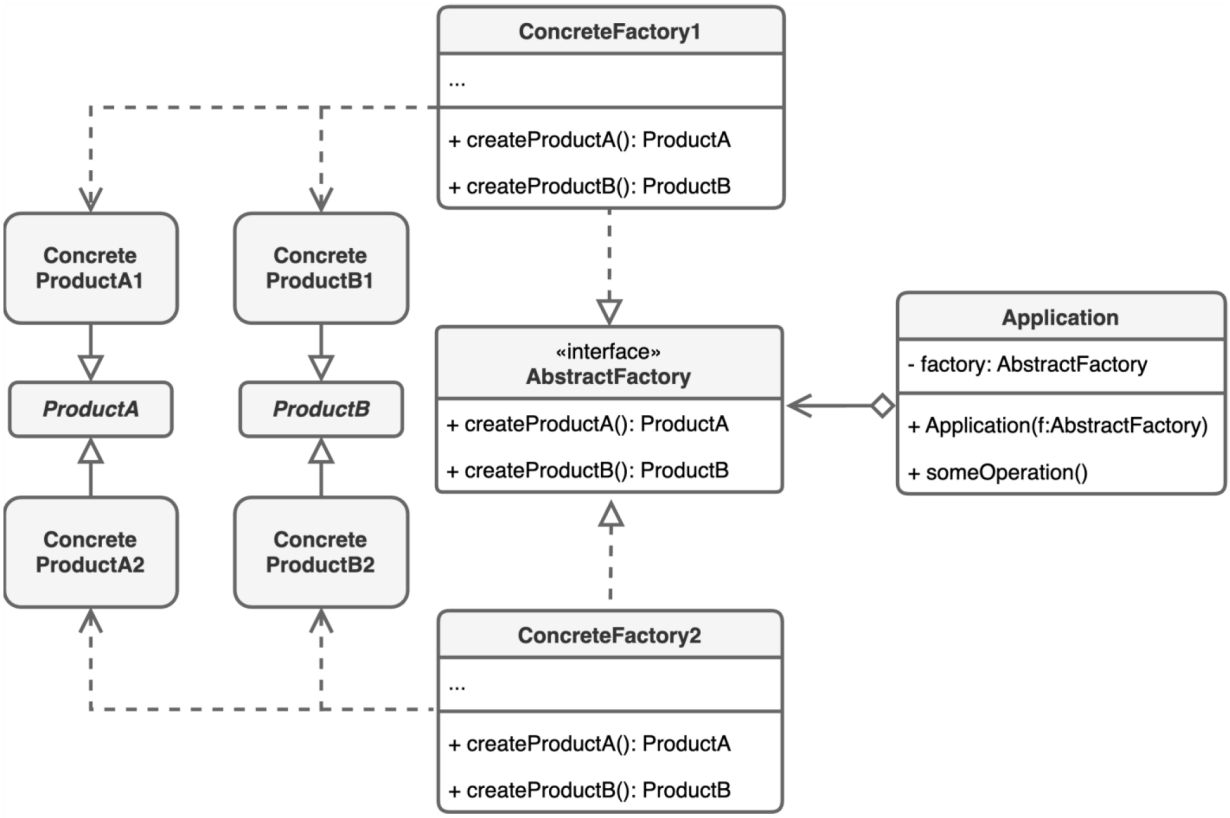

Figure 1—UML diagram of Abstract Factory Design Pattern

Most of the current literature about Abstract Factory design pattern use a slightly adapted Look & Feel example; the original example "consider a user interface toolkit that supports multiple look-and-feel standards" [7] is often represented by the Windows, Mac and Linux GUI interfaces. This example demonstrates the use of Abstract Factory advanced known number of Products such as Buttons, Checkboxes and Fields.

In Figure 2, we leverage the conventional Abstract Factory Design Pattern to efficiently manage multiple compute architectures and the intricate physics of the wave equation. This design pattern allows us to demonstrate the scalability and flexibility required when dealing with complex computational tasks. In this specific example, we illustrate how having only two options for each parameter results in four distinct combinations of classes that the Factory must oversee. The Abstract Factory, referred to as the Kernel Factory in this context, is tasked with managing wave propagation kernels. These kernels are responsible for solving one time step of the wave equation, making them crucial for accurate and efficient simulations. The Kernel Factory outputs a kernel based on the computer architecture of the available hardware and the specified wave physics, ensuring optimal performance and adaptability.

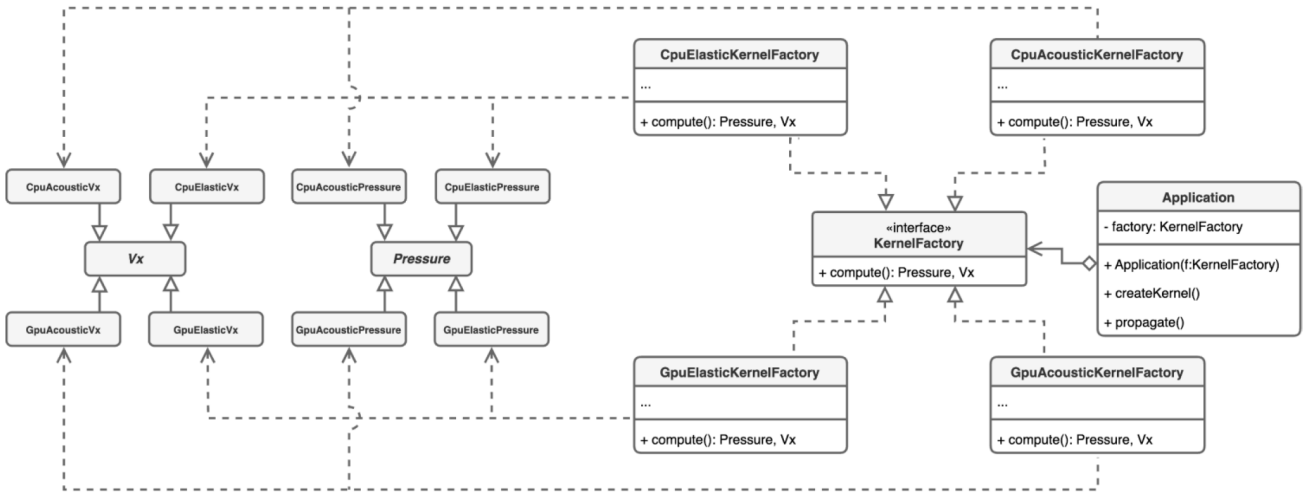

Figure 2—UML diagram of Conventional Abstract Factory Design Pattern implemented to manage multiple compute architectures and wave equation physics

The computer architecture in our setup provides two primary options: a CPU or a GPU. Additionally, the wave physics can be either acoustic or elastic, leading to a total of four different classes that must be manually created: CPU acoustic, CPU elastic, GPU acoustic, and GPU elastic kernels. Each of these kernels is designed to output various data types, such as pressure and vector velocity values, including components like Vx. This structured approach ensures that the correct kernel is selected based on the specific requirements of the simulation, thereby enhancing computational efficiency. However, it is important to note that as the number of parameters and options increases, the number of possible kernels that need to be maintained grows exponentially.

The example provided illustrates the application of the Abstract Factory pattern with a predefined set of products. However, such implementations are rare in real-world scenarios. Predicting the need to introduce additional products or entire families of related and interdependent products is challenging. It is even more difficult to envision a design that can anticipate future changes in specific architectural designs, such as CPU or GPU Application Programming Interfaces (APIs). This underscores the critical role of a robust design pattern like the Abstract Factory Generator in managing complexity and ensuring the scalability of our computational solutions.

Abstract Factory Generator Design Pattern for Seismic Kernels

The Abstract Factory Generator is a sophisticated design pattern that consists of three essential components: the Factory Generator, the Registry, and the Decoder. As depicted in Figure 3, the Factory Generator serves as the central orchestrator, managing a collection of Concrete Factories that are organized around a generic parent Factory. This parent Factory is designed to encapsulate common functionalities that are shared across the various Concrete Factories, thereby promoting code reusability and maintaining a consistent interface. The Registry, a crucial component of this pattern, is a centralized repository that statically stores all Factories along with their respective Concrete variants. It employs a standardized key mapping scheme to efficiently organize and retrieve the Factories, ensuring seamless access and management.

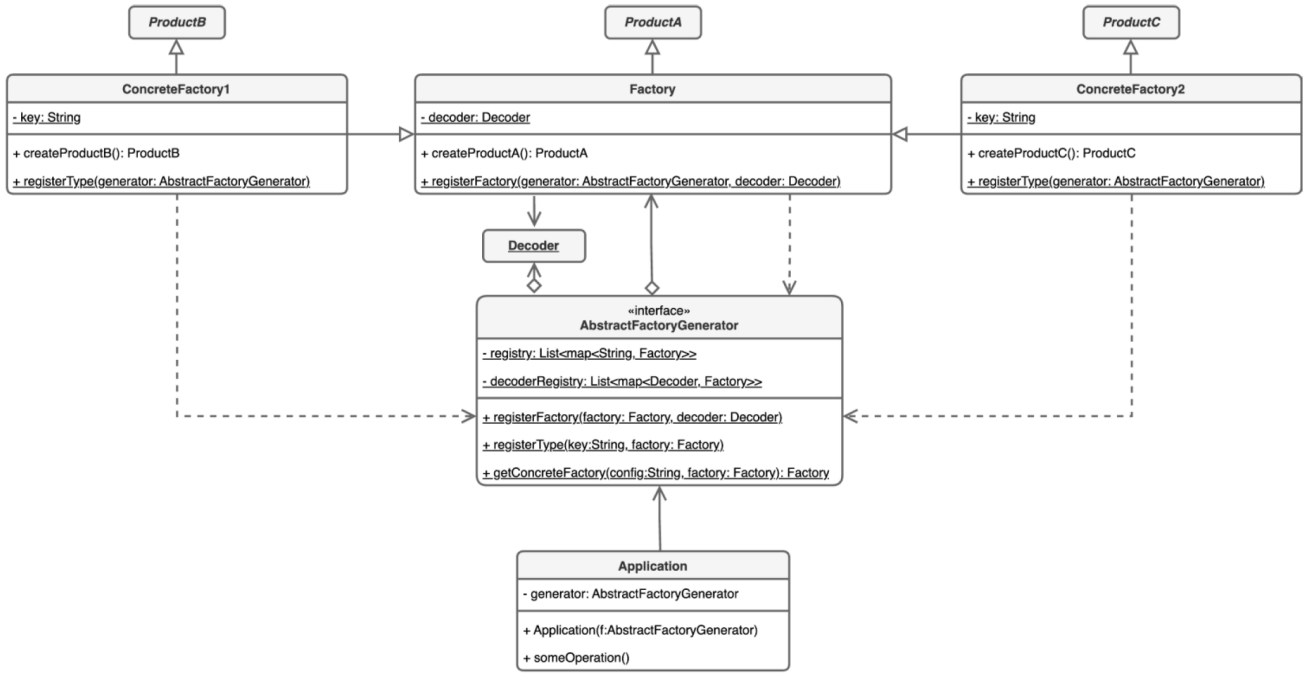

Figure 3—UML diagram of Abstract Factory Generator Design Pattern

The Decoder class plays a vital role in this pattern by parsing user-provided configuration files and interpreting the settings to match the corresponding keys stored within the Registry. This process allows the Decoder to output the appropriate Concrete Factory, facilitating the dynamic creation of objects based on user configurations. The Abstract Factory Generator offers several advantages over the traditional Abstract Factory pattern. It significantly reduces repetitive code between similar Concrete Factories by consolidating all necessary information within the Abstract Factory Generator. Additionally, it eliminates the need for multiple Abstract Factories in applications, as all functionalities are centralized. Furthermore, this pattern enables the addition of new Factories on the fly, leveraging the robust functionalities of the Abstract Factory Generator to accommodate evolving application requirements.

In Figure 4, our application leverages the Abstract Factory Generator Design Pattern to efficiently manage the propagation kernels for wave equations. This pattern ensures consistency and flexibility by maintaining a static Generator interface that oversees all Factories within the application. In this context, the Generator is tasked with managing the Acoustic Kernel Factory class, which is responsible for handling all kernels that share common functionalities. Through the register factory functionality, the Generator facilitates the management of these kernels, which encompass a wide range of architectures and anisotropy combinations. This centralized approach not only simplifies the management process but also enhances the scalability and maintainability of the application.

Furthermore, the Generator manages the hierarchical structure of Factories through the register type functionality. This feature allows for the registration of concrete implementations by assigning a unique key and linking them to a parent registered Factory. Each entry is meticulously stored in the registry, ensuring that all registered implementations are readily accessible. At runtime, the Generator's getter function plays a crucial role by retrieving any registered implementation as needed. This dynamic retrieval capability allows the application to adapt to various requirements and configurations, making it a robust solution for managing multiple complex wave equation propagation kernels as well as other similar functionalities, all in one place.

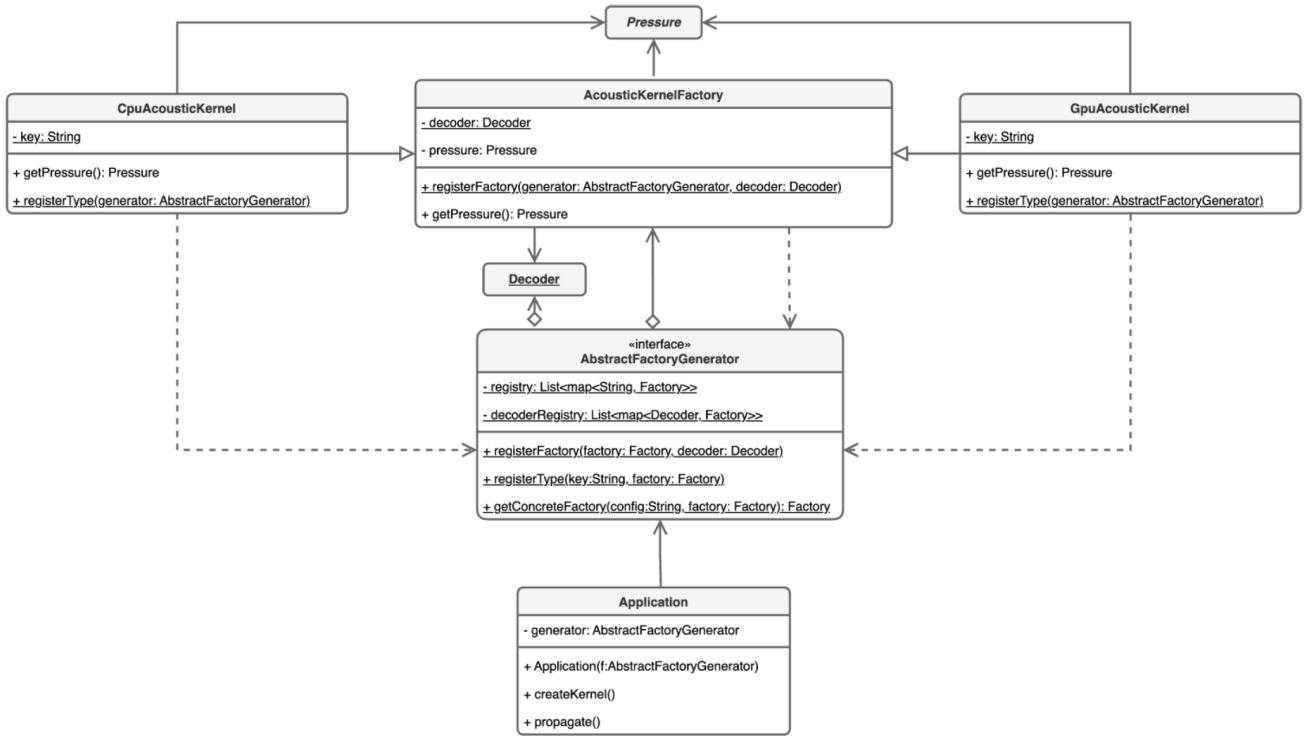

Figure 4—UML diagram of Abstract Factory Generator Design Pattern implemented to manage multiple compute architectures and wave equation physics

Conclusion

The Abstract Factory Generator design pattern offers unparalleled advantages when applied to the management of wave equation propagation kernels, as compared to the conventional Abstract Factory pattern. One of the most significant benefits is its ability to consolidate all factory functionalities within a single, centralized interface. This eliminates the redundancy of creating multiple Abstract Factory modules, thereby streamlining the process of managing complex computational setups like CPU and GPU architectures paired with acoustic or elastic wave physics. Such consolidation not only reduces maintenance overhead but also significantly enhances the scalability of applications, enabling seamless integration of new kernel types or architectures as simulation requirements evolve.

Moreover, the dynamic and flexible nature of the Abstract Factory Generator empowers applications to adapt efficiently to a diverse range of wave equation parameters and configurations. The robust use of the Registry and Decoder ensures that user-defined settings are parsed and matched to the correct factory implementations, facilitating real-time adjustments and object creation. This capability is particularly critical for simulations involving intricate wave physics, where precise alignment of pressure and velocity data outputs is essential. By maintaining a static Generator interface, the pattern ensures consistency across multiple setups while accommodating anisotropy and architectural combinations without compromising computational efficiency.

Finally, the hierarchical structure embedded within the Abstract Factory Generator design pattern adds a layer of sophistication to the management of propagation kernels. The ability to register concrete implementations and retrieve them dynamically at runtime not only simplifies the development process but also ensures long-term sustainability of the application. The enhanced scalability and maintainability offered by this approach make it an ideal solution for the increasingly complex demands of wave equation simulations. As computational architectures continue to evolve, the Abstract Factory Generator stands out as a forward-thinking design choice, capable of addressing both current and future challenges in scientific applications.

References

1. N. Nahar and K. Sakib, "Automatic Recommendation of Software Design Patterns Using Anti-patterns in the Design Phase: A Case Study on Abstract Factory".
2. A. Bulajic and S. Jovanovic, "An Approach to Reducing Complexity in Abstract Factory Design Pattern," vol. **3**, 2012.
3. A. Shvets, *Dive Into Design Patterns*. Refactoring.Guru, 2019.
4. M. Louboutin, G. Gorman, and F.J. Herrmann, "Optimizing the computational performance and maintainability of time-domain modelling—leveraging multiple right-hand-sides".
5. P. A. Witte et al, "A large-scale framework for symbolic implementations of seismic inversion algorithms in Julia," *GEOPHYSICS*, vol. **84**, no. 3, pp. F57–F71, May 2019, doi: 10.1190/geo2018-0174.1.
6. M. Louboutin et al, "Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration," *Geosci. Model Dev.*, vol. **12**, no. 3, pp. 1165–1187, Mar. 2019, doi: 10.5194/gmd-12-1165-2019.
7. E. Gamma, Ed., *Design patterns: elements of reusable object-oriented software*. in Addison-Wesley professional computing series. Reading, Mass: Addison-Wesley, 1995.